

Reviewer Pack — Width-Bounded Module Dimension in Barrington’s Branching Pro...

Entry b01f827fc041 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 07:49:08 UTC

Width-Bounded Module Dimension in Barrington’s Branching Programs

Entry ID: b01f827fc041 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 07:49:08 UTC

1. Conjecture

Field A (mathematical branch): Module Theory over Non-Commutative Rings **Field B** (complexity object): Width-5 Algebraic Branching Programs

Statement:

The minimal dimension of a faithful module over the transition algebra of a width-5 branching program is polynomially bounded by its length, and this bound is tight for functions with high symmetry.

Rationale (proposer’s reasoning):

Module theory over non-commutative rings is rarely applied to complexity theory, yet branching programs’ transition algebras naturally form such structures. Linking their module dimensions to width provides a novel algebraic lens for analyzing NC¹ circuit simulation.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 81aaf7a0978a6e32

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(a, m):
    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        raise ValueError("Modular inverse does not exist")
    else:
        return x % m

def matrix_multiply(A, B):
    rows_A, cols_A = len(A), len(A[0])
    rows_B, cols_B = len(B), len(B[0])
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_add(A, B):
    rows = len(A)
```

```

cols = len(A[0])
result = [[A[i][j] + B[i][j] for j in range(cols)] for i in range(rows)]
return result

def matrix_subtract(A, B):
    rows = len(A)
    cols = len(A[0])
    result = [[A[i][j] - B[i][j] for j in range(cols)] for i in range(rows)]
    return result

def transpose_matrix(matrix):
    rows = len(matrix)
    cols = len(matrix[0])
    result = [[matrix[j][i] for j in range(rows)] for i in range(cols)]
    return result

def matrix_power(matrix, n):
    if n == 1:
        return matrix
    elif n % 2 == 0:
        half_power = matrix_power(matrix, n // 2)
        return matrix_multiply(half_power, half_power)
    else:
        return matrix_multiply(matrix, matrix_power(matrix, n - 1))

def gaussian_elimination(A):
    rows, cols = len(A), len(A[0])
    for i in range(rows):
        max_row = i
        for j in range(i+1, rows):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        factor = A[i][i]
        for j in range(cols):
            A[i][j] /= factor
        for j in range(rows):
            if i != j:
                factor = A[j][i]
                for k in range(cols):
                    A[j][k] -= factor * A[i][k]
    return A

def compute_transition_algebra(program, q):
    n = len(q)
    transition_matrix = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if program[i][j] != 0:
                transition_matrix[i][j] = 1
    return transition_matrix

def compute_module_dimension(transition_matrix):
    rows, cols = len(transition_matrix), len(transition_matrix[0])
    augmented_matrix = [row + [1] for row in transition_matrix]
    reduced_matrix = gaussian_elimination(augmented_matrix)

```

```

rank = sum(1 for row in reduced_matrix if any(row[i] != 0 for i in range(cols)))
return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 5
    q = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    transition_matrix = compute_transition_algebra(q, q)
    module_dimension = compute_module_dimension(transition_matrix)
    length = sum(sum(row) for row in q)
    symmetry = sum(q[i][j] == q[j][i] for i in range(n) for j in range(i+1, n))
    metric_value = module_dimension / (length + 1e-9)
    instances_tested = 1
    conjecture_holds = False
    counterexample = ""
    return {
        "metric_name": "module_dimension",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8c8cf36.py", line 137, in <module>
    result = run_trial(seed)

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8c8cf36.py", line 117, in run_trial
    module_dimension = compute_module_dimension(transition_matrix)
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8c8cf36.py", line 108, in compute_modu
    reduced_matrix = gaussian_elimination(augmented_matrix)
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8c8cf36.py", line 88, in gaussian_elim
    A[i][j] /= factor
ZeroDivisionError: float division by zero

```

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b01f827fc041.tar.gz (if generated)